0049. Group Anagrams

Medium · Arrays & Hashing

leetcode.com/problems/group-anagrams

Problem

- Given an array of strings, group the anagrams together. Any order of groups is accepted.
- Example: ["eat","tea","tan","ate","nat","bat"] → [["bat"],["nat","tan"],["ate","eat","tea"]].

Intuition

- All anagrams share a canonical "signature". If we compute it, a HashMap from signature to list does the grouping.
- Signature options: the sorted word (O(k log k)) or a 26-letter count encoding (O(k)).

Brute Force

- Compare every word with every existing group using an anagram check: O(n^2 k).

Optimal Approach

- For each word, build a count array of 26 letters.
- Encode the counts as a key like "#1#0#0...#1" (separators avoid ambiguity such as 1,11 vs 11,1).
- ComputeIfAbsent(key, new list).add(word).
- Return the map's values.

Worked Example

```
words = ["eat","tea","tan","ate","nat","bat"], key shown as letters that occur.

eat -> a1 e1 t1  -> group A [eat]
tea -> a1 e1 t1  -> group A [eat,tea]
tan -> a1 n1 t1  -> group B [tan]
ate -> a1 e1 t1  -> group A [eat,tea,ate]
nat -> a1 n1 t1  -> group B [tan,nat]
bat -> a1 b1 t1  -> group C [bat]
```

Java Solution

```java
public class GroupAnagrams {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> groups = new HashMap<>();
        for (String w : strs) {
            int[] count = new int[26];
            for (char c : w.toCharArray()) count[c - 'a']++;
            StringBuilder key = new StringBuilder();
            for (int c : count) key.append('#').append(c);
            groups.computeIfAbsent(key.toString(), k -> new ArrayList<>()).add(w);
        }
        return new ArrayList<>(groups.values());
    }
}
```

Complexity

- Time: O(n k) where k is the max word length.
- Space: O(n k) for keys and groups.

Pitfalls

- Using a count key without separators can collide.
- Int[] cannot be a HashMap key directly (identity hashing); convert it to a String or a List.